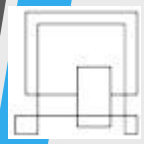
VIA University College



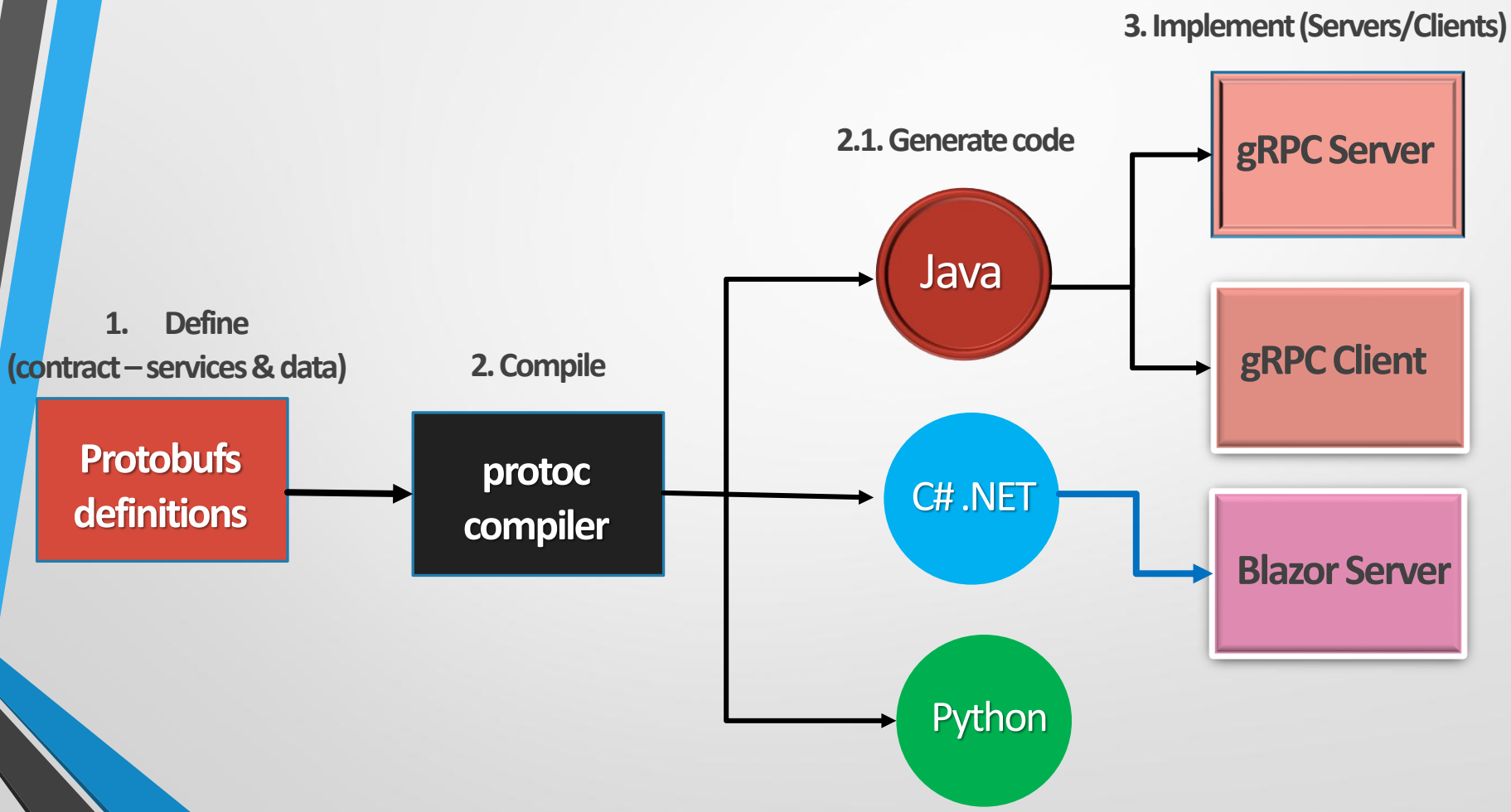
Programming 3
Distributed Systems

Autumn 2026

gRPC and REST



gRPC Workflow



gRPC and REST (I)

- REST uses HTTP/1.x and separate TCP connection per request.
- REST uses text for transport.
- REST is not domain-specific.
- REST is not strongly-typed
- REST lacks support for streaming

gRPC and REST (II)

- gRPC transfer binary data instead of text thus, making communication much faster.
- gRPC is roughly 7 times faster than REST when receiving data and 10 times faster when sending data.*
- gRPC use HTTP/2 with long-lived connections avoiding the overhead of making connection per request.
- gRPC is type-safe so if you are expecting an integer then nobody can send it as a string.
- gRPC supports streaming (distributed streaming)

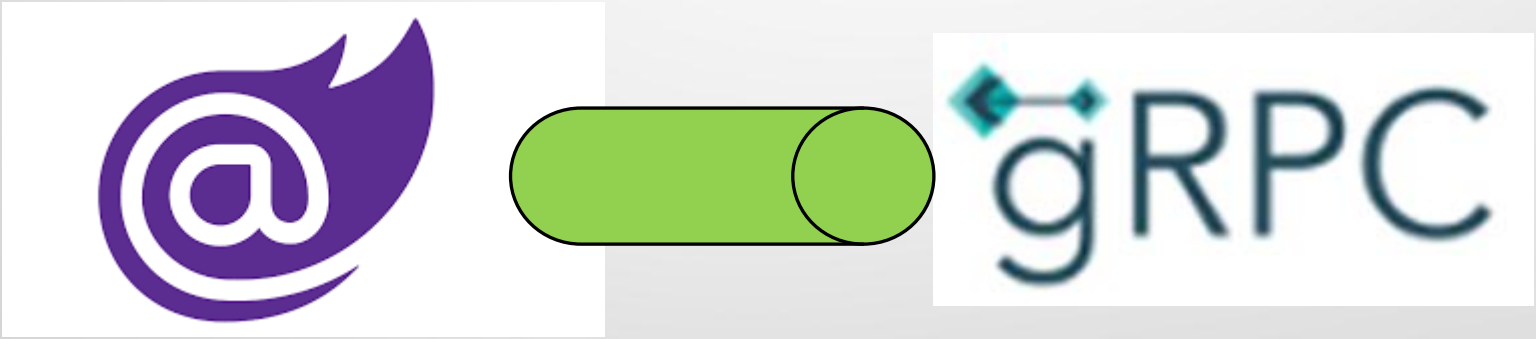
https://medium.com/@EmperorRXF/evaluating-performance-of-rest-vs-grpc-1b8bdf0b22da#:~:text=gRPC%20is%20roughly%207%20times,of%20HTTP%2F2%20by%20gRPC.

gRPC Spring Boot Integration

- Spring boot gRPC framework is run on any environment; it will enable client and server applications to communicate with each other transparently.
- Spring boot gRPC is basically used to develop a mobile application that was used to communicate with a cloud server.
- Spring boot gRPC supports the extension for layered design like logging, load balancing, and monitoring.
- Use @GrpcService to automatically create and run a gRPC service, embedded in the spring-boot application

Inter-Service Communication

- gRPC supports inter-service communication across the intranet and internet and calls from the mobile app and web browser to the gRPC server.



gRPC vs REST using JSON

Feature	gRPC	HTTP using JSON/XML
Language independent	Yes	Yes
HTTP version	HTTP/2	HTTP 1.1
Specifying Domain Schema	.proto files (Google Protocol Buffer)	None
Serialized data size	Least	High (higher for XML)
Human Readable	No, as it uses separate encoding schema	Yes, as it uses text based format
Serialization speed	Fastest	Slower (slowest for XML)
Data type support	Richer. Supports complex data types like Any, one of etc.	Supports basic data types
Support for evolving schema	Yes	No

Designing REST/gRPC API

REST:

```
POST
/api/foo

GET
/api/foo/{foo_id}
```

gRPC:

```
rpc AddFoo(Foo)
    returns FooID;

rpc GetFoo(FooID)
    returns Foo;
```

Designing REST/gRPC API

REST:

```
POST
/api/foo

GET
/api/foo/{foo_id}
```

gRPC:

```
message Foo {
     FooID id = 1;
     repeated Bar bars = 2;
}

message FooID {
     string val = 1;
}
```

Designing REST/gRPC API

REST:

```
POST
/api/foo/{foo_id}/bar

GET
/api/foo/{foo_id}/bar/
{bar_id}
```

gRPC:

```
rpc AddBar(Bar)
    returns BarID;

rpc GetFoo(BarID)
    returns Bar;
```

Designing REST/gRPC API

REST:

```
POST
/api/foo/{foo_id}/bar

GET
/api/foo/{foo_id}/bar/
{bar_id}
```

gRPC:

```
rpc AddBar(Bar)
    returns BarID;

rpc GetFoo(BarID)
    returns Bar;
```

Designing REST/gRPC API

REST:

```
POST
/api/foo/{foo_id}/bar

GET
/api/foo/{foo_id}/bar/
{bar_id}
```

gRPC:

```
message Bar{
     BarID id = 1;
     int baz = 2;
}

message BarID {
     FooID foo_id = 1;
     string bar_id = 2;
}
```

Designing a gRPC API

REST:

```
POST
/api/foo/{foo_id}/bar

GET
/api/foo/{foo_id}/bar/
{bar_id}
```

gRPC:

```
message Bar{
     BarID id = 1;
     int baz = 2;
}

message BarID {
     FooID foo_id = 1;
     string bar_id = 2;
}
```

Designing a gRPC API

REST:

```
POST
/api/foo/{foo_id}/bar

GET
/api/foo/{foo_id}/bar/
{bar_id}
```

gRPC:

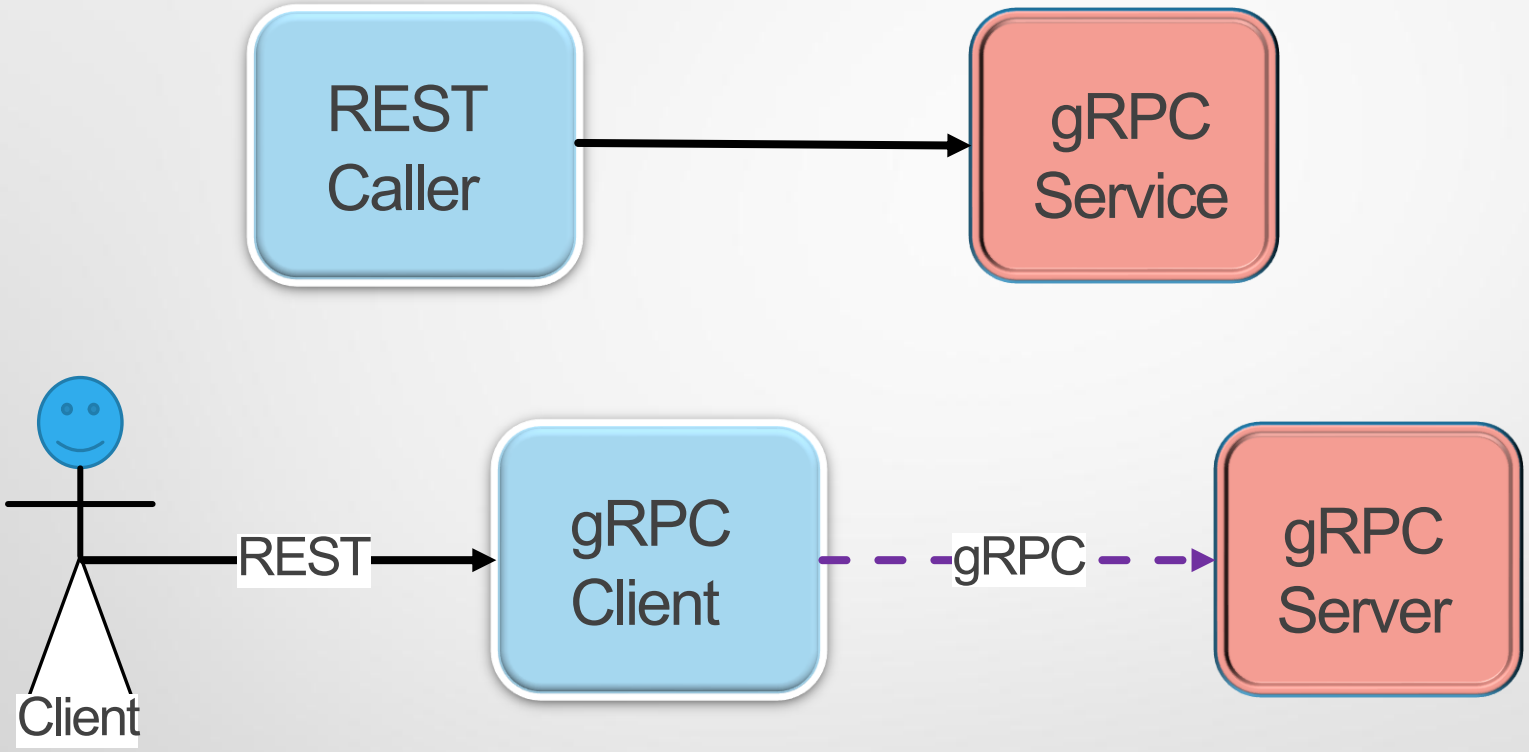
```
message Bar{
     BarID id = 1;
     int baz = 2;
}

message BarID {
     FooID foo_id = 1;
     string bar_id = 2;
}
```

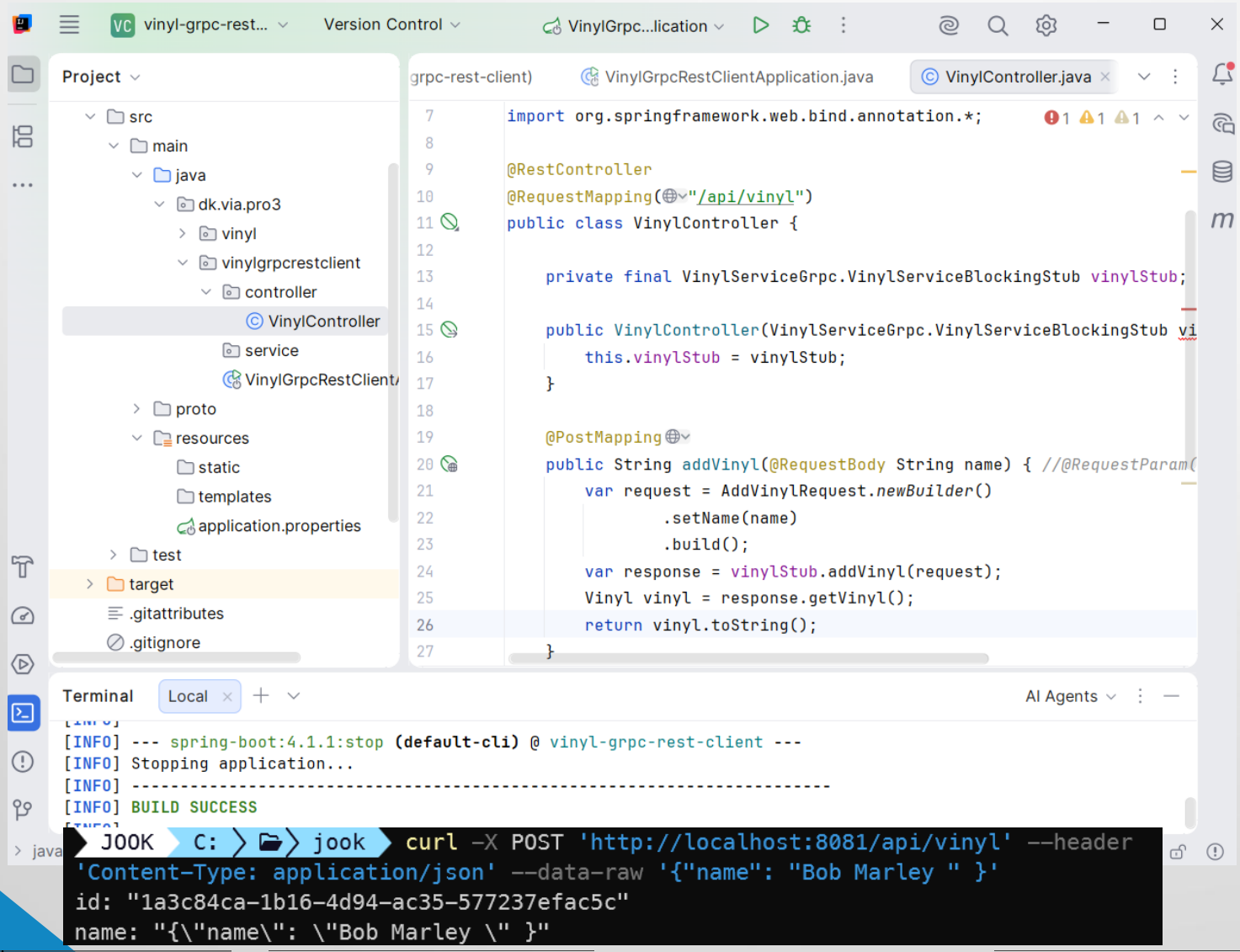
Live Coding/Hands-on/Demo test the REST API (getById())



Serve REST and GRPC

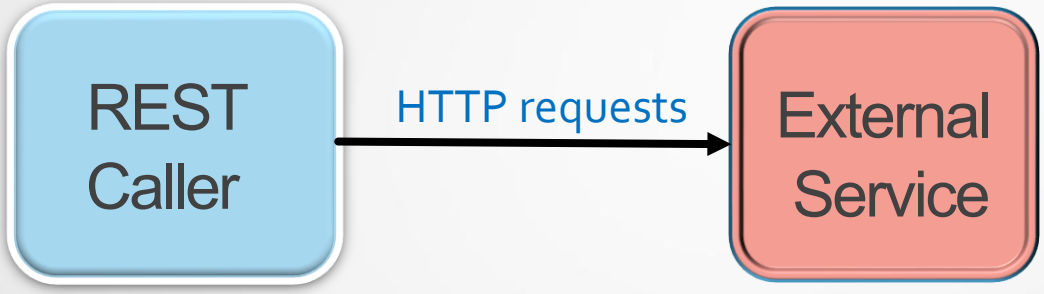


Example/Demo



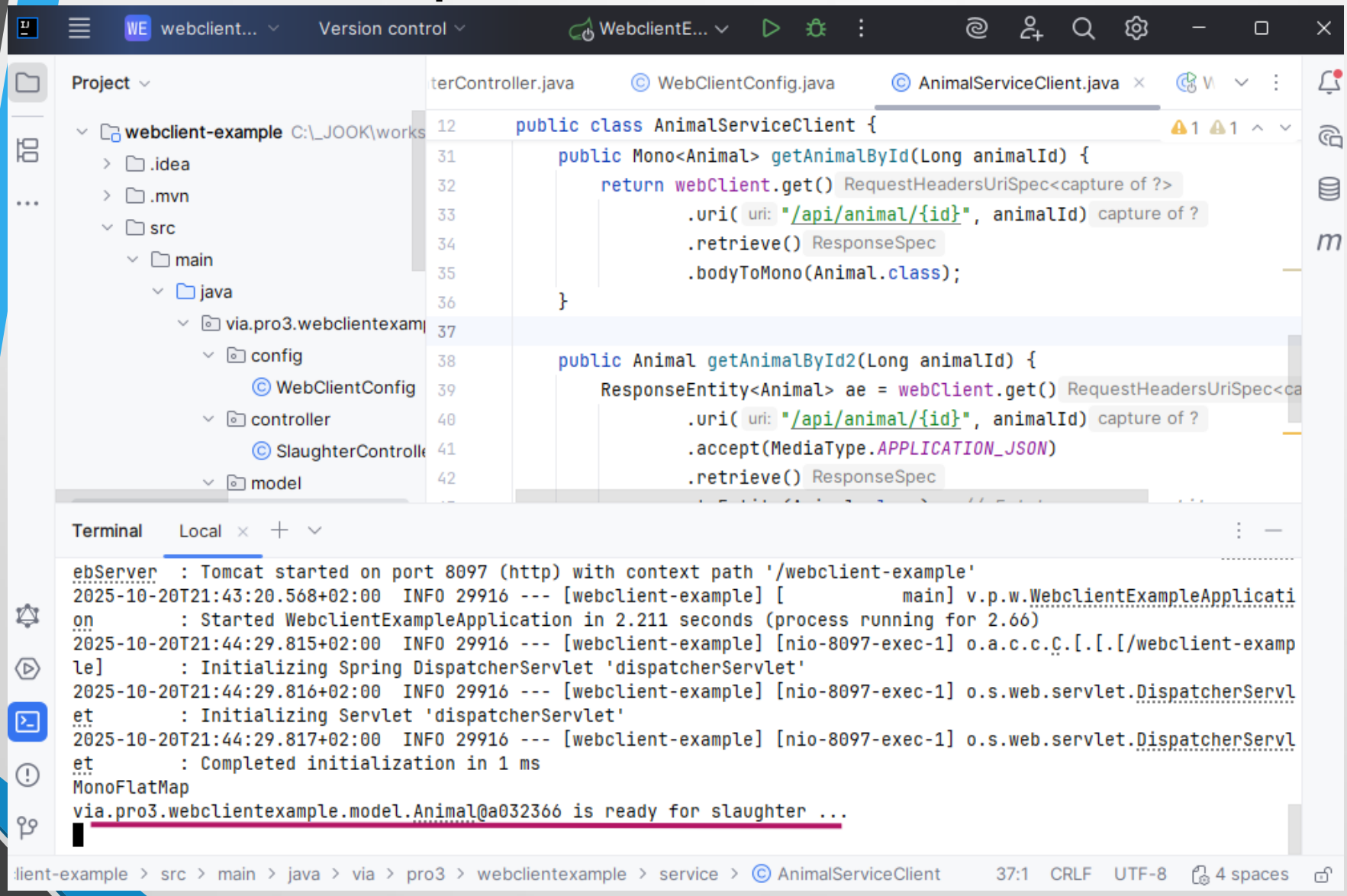
Calling REST APIs

- Making HTTP requests to external services
- Calling REST APIs from a Spring Boot Application

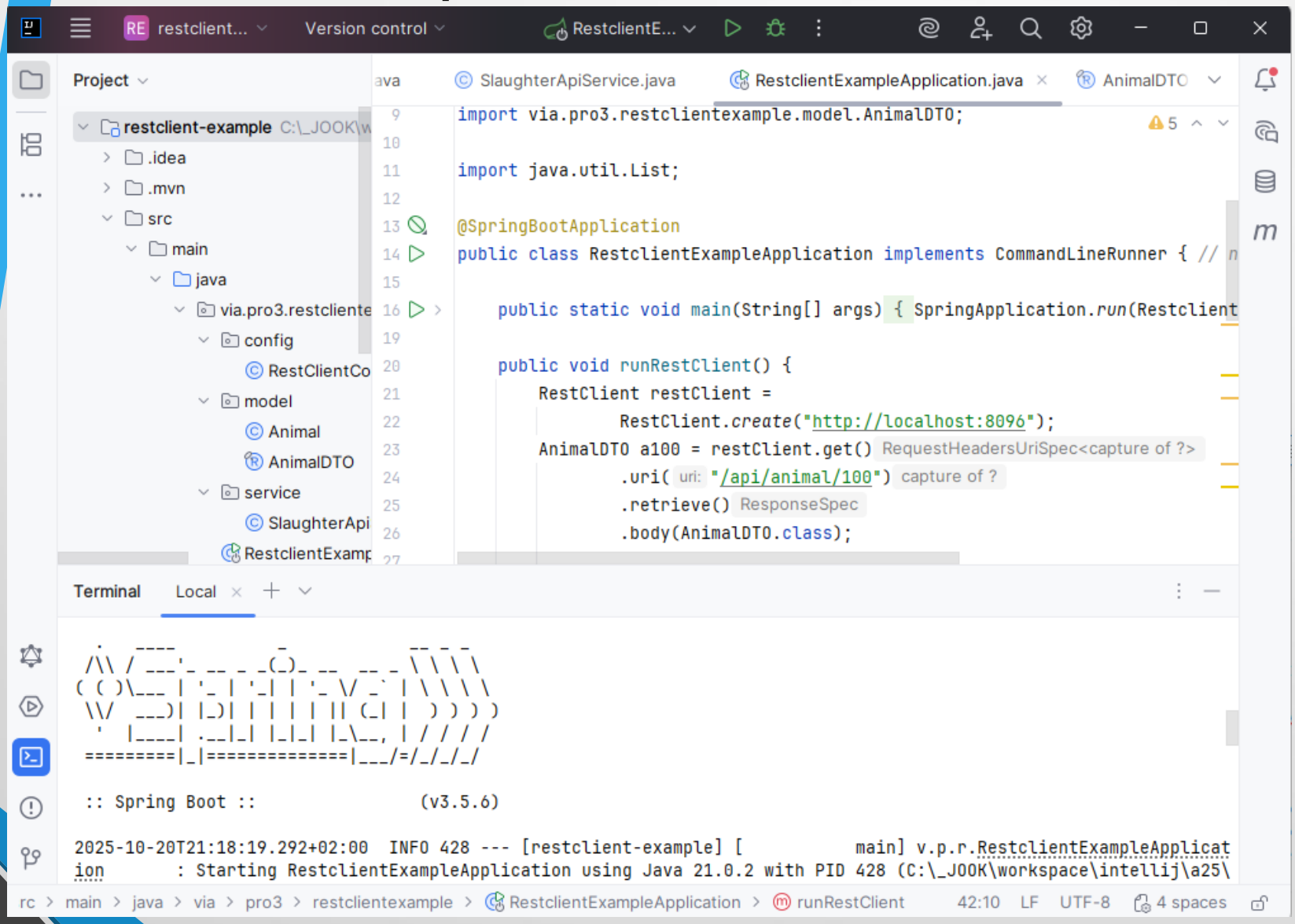


- RestClient
  - Builds upon WebClient to give a more intuitive and modern approach to consuming RESTful services.
  - supports the modern HTTP standards such as HTTP/2
- WebClient
  - Non-blocking and reactive web client to perform HTTP requests
  - A replacement for RestTemplate

Example/Demo-WebClient



Example/Demo-RestClient



Microservices

What are Microservices?

- Monolithic Applications
  - Codebase of application deployed as a single bundle of executables and libraries on a unified platform
- Microservices Architecture
  - Multiple independent software components orchestrated to form a unified application
  - Common infrastructure:
    - User interface toolkit
    - API Gateway
    - Persistence layer

Who is using Microservices?

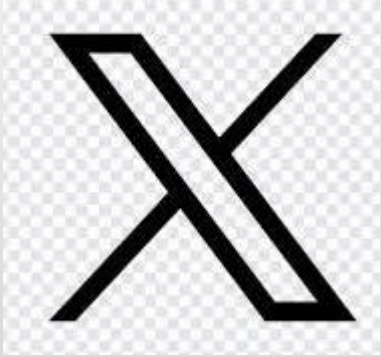
- Uber
- Netflix
- Amazon
- Ebay
- Twitter
- PayPal
- etc.













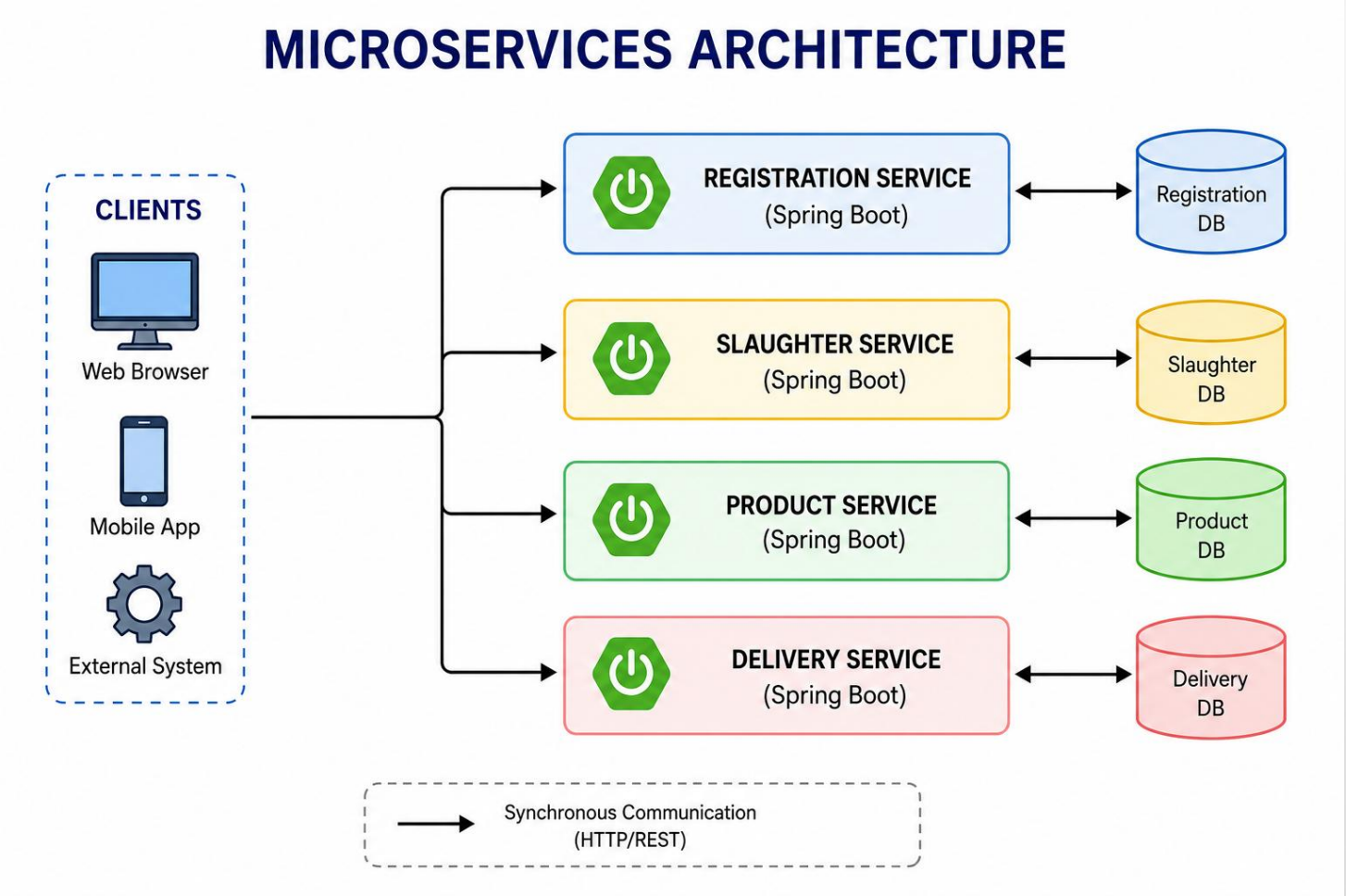
Building a microservice

- Small unit of functionality
- Complete and independent technology stack
- Separate data stores
  - Synchronization with other services as needed through persistence layers
- Invoked through API Request / Response
- Usually: REST, HTTP, JSON
- Self-contained components
- Inner workings not exposed externally
  - Developers have free reign to select tech components

Assembling an application

- Each microservice performs a small limited task
  - Not intended to be standalone applications
- Surrounded by specialized infrastructure
- Manage communications
- Orchestrate services into complex chains of tasks
- Load balancing
- API Gateway
- User Interface layer
- Externally exposed APIs
- Multiple instances of any microservice launched as needed

Microservices Architecture (v1)



Live Coding/Hands-on/Demo download the project

